

SID4: 1066

PORT_BASE: 8166

PREFIX: s1066

SEED: 1066

VERIFY_SEED: 261066

DOMAIN_ID: 2

Hardware: Macintosh HD, Apple M4, 24 GB, Tahoe 26.6.2

Local Model Used: Ollama qwen3:8b

Tagged Commit Hash:

<https://github.com/paknathan/data260-1066>

1) Routes

Home, Login, & Dashboard Page

Index.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1" />
6      <title>FastAPI Homework 3</title>
7      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
8  </head>
9  <body>
10
11  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
12      <div class="container">
13          <a class="navbar-brand" href="/">FastAPI Homework 3</a>
14          <div class="navbar-nav">
15              {% if user %}
16                  <a class="nav-link" href="/dashboard">Dashboard</a>
17                  <a class="nav-link" href="/logout">Logout</a>
18              {% else %}
19                  <a class="nav-link" href="/login">Login</a>
20              {% endif %}
21          </div>
22      </div>
23  </nav>
24
25  <div class="container mt-4">
26      <div class="card">
27          <div class="card-body">
28              <h3 class="card-title">Welcome to Municipal Transit Incidents</h3>
29
30              {% if user %}
31                  <p class="card-text">You are logged in as <strong>{{ user }}</strong>.</p>
32                  <a class="btn btn-primary" href="/dashboard">Go to Dashboard</a>
33                  <a class="btn btn-outline-danger ms-2" href="/logout">Logout</a>
34              {% else %}
35                  <p class="card-text">Please log in to continue.</p>
36                  <a class="btn btn-success" href="/login">Login</a>
37              {% endif %}
38          </div>
39      </div>
40  </div>
41
42  </body>
43  </html>
44

```

```

1  #auth.py
2  from fastapi import APIRouter, Request, Form
3  from fastapi.responses import RedirectResponse, HTMLResponse
4  from fastapi.templating import Jinja2Templates
5  from starlette.status import HTTP_302_FOUND
6
7
8  # Create a router object
9  # This behaves like a mini FastAPI app
10 router = APIRouter()
11
12 # Configure Jinja2 templates directory
13 templates = Jinja2Templates(directory="templates")
14
15
16 # Hardcoded credentials for demo purposes only
17 # In real applications, credentials come from a database
18 VALID_USERNAME = "admin"
19 VALID_PASSWORD = "password"
20
21
22 @router.get("/", response_class=HTMLResponse)
23 def home(request: Request):
24     """
25     Home page route.
26
27     - Checks if a user is logged in using the session
28     - Passes user info to the template
29     """
30     user = request.session.get("user")
31
32     return templates.TemplateResponse(
33         "index.html",
34         {
35             "message": "Welcome Home!",
36             "request": request,
37             "user": user
38         }
39     )
40
41 @router.get("/login")
42 def login_page(request: Request):
43     """
44     Displays the login form.
45
46     If the user is already logged in,
47     the template can choose what to display.
48     """
49     user = request.session.get("user")
50     error = request.session.pop("error", None)
51
52     return templates.TemplateResponse(

```

```

50     error = request.session.pop("error", None)
51
52     return templates.TemplateResponse(
53         "login.html",
54         {
55             "request": request,
56             "user": user,
57             "error": error
58         }
59     )
60
61
62 @router.post("/login")
63 def login(request: Request, username: str = Form(...), password: str = Form(...)):
64     """
65     Handles login form submission.
66
67     - Reads username and password from the form
68     - Validates credentials
69     - Stores user info in session if valid
70     """
71     if username == VALID_USERNAME and password == VALID_PASSWORD:
72         # Store logged-in user in session
73         request.session["user"] = username
74
75         # Redirect user to dashboard
76         return RedirectResponse(
77             url="/dashboard",
78             status_code=HTTP_302_FOUND
79         )
80
81     # If credentials are invalid:
82     # Redirect back to login page
83     #
84     # NOTE:
85     # No error message is shown intentionally.
86     # Students are expected to add Bootstrap alerts.
87     request.session["error"] = "Invalid username or password."
88
89     return RedirectResponse(
90         url="/login",
91         status_code=HTTP_302_FOUND
92     )
93
94
95 @router.get("/dashboard")
96 def dashboard(request: Request):
97     """
98     Protected route.
99
100     - Only accessible if user is logged in
101     - Redirects to login page if session is missing
102     """

```

```

100     - Only accessible if user is logged in
101     - Redirects to login page if session is missing
102     """
103     user = request.session.get("user")
104
105     # If user is not logged in, block access
106     if not user:
107         return RedirectResponse(
108             url="/login",
109             status_code=HTTP_302_FOUND
110         )
111
112     # If user is logged in, render dashboard
113     return templates.TemplateResponse(
114         "dashboard.html",
115         {
116             "request": request,
117             "user": user
118         }
119     )
120
121
122 @router.get("/logout")
123 def logout(request: Request):
124     """
125     Logs the user out.
126
127     - Clears all session data
128     - Redirects back to home page
129     """
130     request.session.clear()
131
132     return RedirectResponse(
133         url="/",
134         status_code=HTTP_302_FOUND
135     )
136

```

```

1  #main.py
2  from fastapi import FastAPI, Form, Request
3  from fastapi.responses import HTMLResponse, RedirectResponse
4  from fastapi.templating import Jinja2Templates
5  from routers.auth import router
6  import re
7  import uvicorn
8  from starlette.middleware.sessions import SessionMiddleware
9
10 app = FastAPI(title="Municipal Transit Incident API")
11
12 app.add_middleware(
13     SessionMiddleware,
14     secret_key="data260-session-secret")
15
16 app.include_router(router)
17
18 templates = Jinja2Templates(directory="templates")
19
20 incidents_db = [
21     {
22         "incident_id": "INC-00147",
23         "route_or_line": "Route 22",
24         "incident_type": "Mechanical Breakdown",
25         "email": "operator@transit.org",
26         "description": "Engine failure near 4th street stop."
27     }
28 ]
29
30 @app.get("/", response_class=HTMLResponse)
31 def read_root(request: Request, updated: str | None = None, q: str | None = None):
32     search_query = (q or "").strip()
33
34     if search_query:
35         needle = search_query.lower()
36         filtered_incidents = [
37             incident for incident in incidents_db
38             if needle in incident["route_or_line"].lower()
39             or needle in incident["incident_type"].lower()
40         ]
41     else:
42         filtered_incidents = incidents_db
43
44     return templates.TemplateResponse(
45         "index.html",
46         {
47             "request": request,
48             "incidents": filtered_incidents,
49             "updated_id": updated,
50             "search_query": search_query,
51             "has_incidents": bool(incidents_db),
52         }
53     )
54

```

```

50         "search_query": search_query,
51         "has_incidents": bool(incidents_db),
52     }
53 )
54
55 @app.post("/add")
56 def add_incident(
57     incident_id: str = Form(...),
58     route_or_line: str = Form(...),
59     incident_type: str = Form(...),
60     email: str = Form(...),
61     description: str = Form(...)
62 ):
63     new_record = {
64         "incident_id": incident_id,
65         "route_or_line": route_or_line,
66         "incident_type": incident_type,
67         "email": email,
68         "description": description
69     }
70     incidents_db.append(new_record)
71
72     # 303 Redirect back to GET home view
73     return RedirectResponse(url="/", status_code=303)
74
75 @app.post("/update/{incident_id}")
76 def update_incident(
77     incident_id: str,
78     route_or_line: str = Form(...),
79     incident_type: str = Form(...)
80 ):
81     # Update the primary (route/line) and secondary (incident type) fields
82     # for the matching record with whatever values were submitted.
83     for incident in incidents_db:
84         if incident["incident_id"] == incident_id:
85             incident["route_or_line"] = route_or_line
86             incident["incident_type"] = incident_type
87             break
88
89     # Redirect back to GET home view, flagging which record just changed
90     return RedirectResponse(url=f"?updated={incident_id}", status_code=303)
91
92 def _numeric_id(incident_id: str) -> int:
93     """Extract the numeric portion of an incident_id like 'INC-00147' -> 147.
94     Records with no digits at all sort lowest (treated as -1)."""
95     digits = re.findall(r"\d+", incident_id)
96     return int("".join(digits)) if digits else -1
97
98 @app.post("/delete-highest")
99 def delete_highest_incident():
100     if incidents_db:
101         highest = max(incidents_db, key=lambda incident: _numeric_id(incident["incident_id"]))
102         incidents_db.remove(highest)
103
104     # Redirect back to GET home view, which re-renders the (now shorter) list
105     return RedirectResponse(url="/", status_code=303)
106
107 if __name__ == "__main__":
108     uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True)

```

Home, Dashboard, Login, Logout Demo:

FastAPI Homework 3

Login

Welcome to Municipal Transit Incidents

Please log in to continue.

Login

FastAPI Homework 3

Dashboard Logout

Welcome to Municipal Transit Incidents

You are logged in as **admin**.

Go to Dashboard

Logout

FastAPI Homework 3

Login

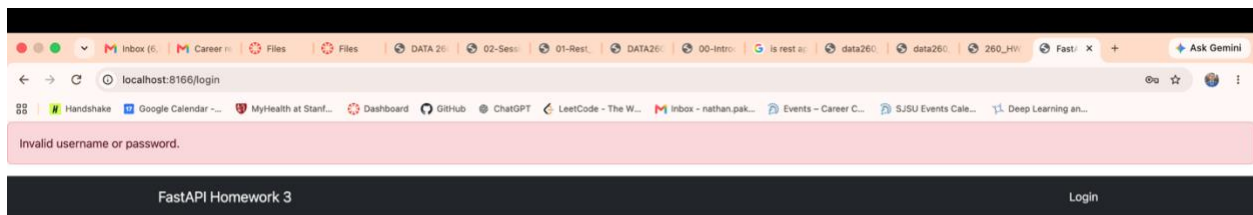
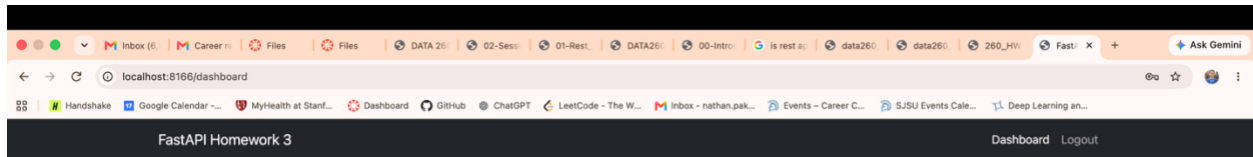
Login

Username

Password

Login

[Back to Home](#)



Login

Username

Password

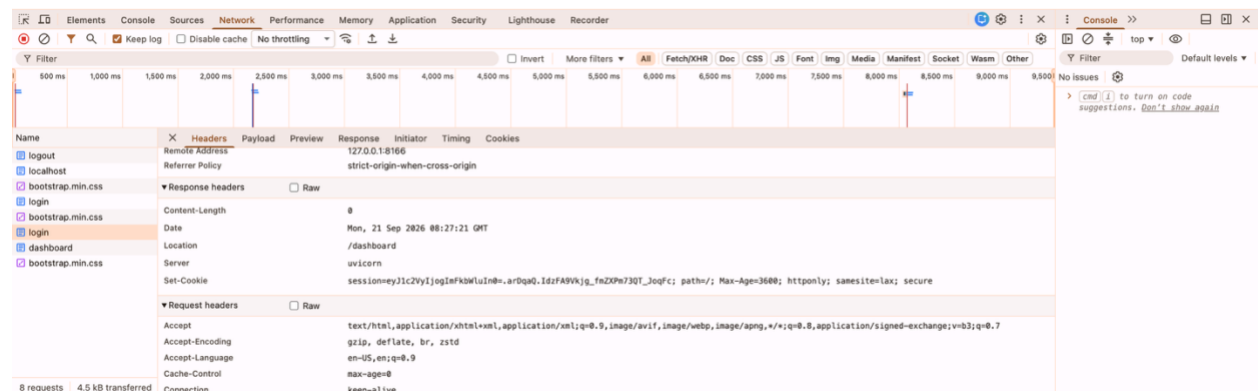
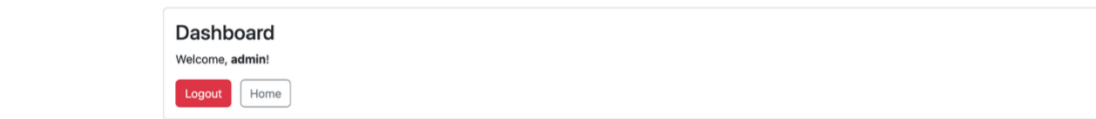
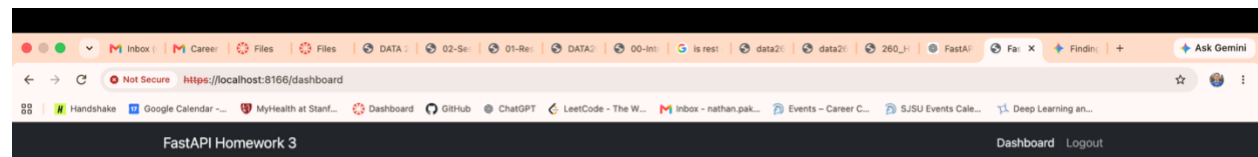
Login

[Back to Home](#)

Session cookie is Secure

```
11
12 app.add_middleware(
13     SessionMiddleware,
14     secret_key="data260-session-secret",
15     https_only=True,
16     same_site="lax",
17     max_age=3600)
18
```

```
110 if __name__ == "__main__":
111     uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True, ssl_keyfile="key.pem", ssl_certfile="cert.pem")
```



Idle Session Timeout: After waiting for more than 60 seconds in the dashboard page after logging in, I attempted to access /dashboard and it redirected me to the login page as shown below.

```
@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """
    Handles login form submission.

    - Reads username and password from the form
    - Validates credentials
    - Stores user info in session if valid
    """
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        # Store logged-in user in session
        request.session["user"] = username
        request.session["last_activity"] = time.time()

        # Redirect user to dashboard
        return RedirectResponse(
            url="/dashboard",
            status_code=HTTP_302_FOUND
        )

    # If credentials are invalid:
    # Redirect back to login page
    #
    # NOTE:
    # No error message is shown intentionally.
    # Students are expected to add Bootstrap alerts.
    request.session["error"] = "Invalid username or password."

    return RedirectResponse(
        url="/login",
        status_code=HTTP_302_FOUND
    )
```

```

@router.get("/dashboard")
def dashboard(request: Request):
    """
    Protected route.

    - Only accessible if user is logged in
    - Redirects to login page if session is missing
    """
    user = request.session.get("user")
    last_activity = request.session.get("last_activity")
    # If user is not logged in, block access
    if not user:
        return RedirectResponse(
            url="/login",
            status_code=HTTP_302_FOUND
        )

    # Session has expired due to inactivity
    if last_activity is None or time.time() - last_activity > IDLE_TIMEOUT:
        request.session.clear()

        return RedirectResponse(
            url="/login",
            status_code=HTTP_302_FOUND
        )

    # User is still active, so update last activity time
    request.session["last_activity"] = time.time()

    # If user is logged in, render dashboard
    return templates.TemplateResponse(
        "dashboard.html",
        {
            "request": request,
            "user": user
        }
    )

```

Login

Username

Password

Login

[Back to Home](#)

▼ templates

login.html

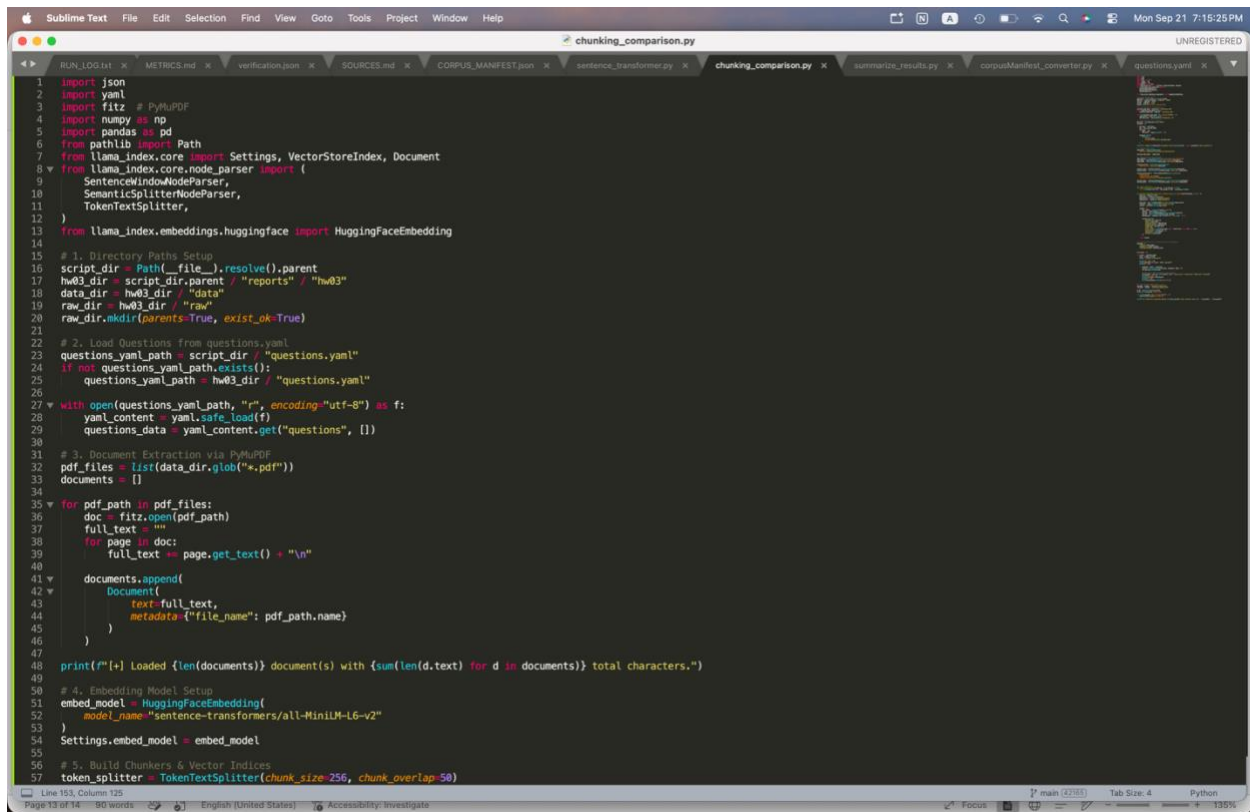
index.html

index_hw2.html

dashboard.html

Part 2: Compare Three LlamaIndex chunking techniques

Chunking_comparison.py



```
1 import json
2 import yaml
3 import fitz # PyMuPDF
4 import numpy as np
5 import pandas as pd
6 from pathlib import Path
7 from llama_index.core import Settings, VectorStoreIndex, Document
8 from llama_index.core.node_parser import (
9     SentenceWindowNodeParser,
10     SemanticSplitterNodeParser,
11     TokenTextSplitter,
12 )
13 from llama_index.embeddings.huggingface import HuggingFaceEmbedding
14
15 # 1. Directory Paths Setup
16 script_dir = Path(__file__).resolve().parent
17 hw03_dir = script_dir.parent / "reports" / "hw03"
18 data_dir = hw03_dir / "data"
19 raw_dir = hw03_dir / "raw"
20 raw_dir.mkdir(parents=True, exist_ok=True)
21
22 # 2. Load Questions from questions.yaml
23 questions_yaml_path = script_dir / "questions.yaml"
24 if not questions_yaml_path.exists():
25     questions_yaml_path = hw03_dir / "questions.yaml"
26
27 with open(questions_yaml_path, "r", encoding="utf-8") as f:
28     yaml_content = yaml.safe_load(f)
29     questions_data = yaml_content.get("questions", [])
30
31 # 3. Document Extraction via PyMuPDF
32 pdf_files = list(data_dir.glob("*.pdf"))
33 documents = []
34
35 for pdf_path in pdf_files:
36     doc = fitz.open(pdf_path)
37     full_text = ""
38     for page in doc:
39         full_text += page.get_text() + "\n"
40
41 documents.append(
42     Document(
43         text=full_text,
44         metadata={"file_name": pdf_path.name}
45     )
46 )
47
48 print(f"Loaded {len(documents)} document(s) with {sum(len(d.text) for d in documents)} total characters.")
49
50 # 4. Embedding Model Setup
51 embed_model = HuggingFaceEmbedding(
52     model_name="sentence-transformers/all-MiniLM-L6-v2"
53 )
54 Settings.embed_model = embed_model
55
56 # 5. Build Chunkers & Vector Indices
57 token_splitter = TokenTextSplitter(chunk_size=256, chunk_overlap=50)
```

```
Sublime Text  File  Edit  Selection  Find  View  Goto  Tools  Project  Window  Help  chunking_comparison.py  UNREGISTERED
RUN_LOG.txt  METRICS.md  verification.json  SOURCES.md  CORPUS_MANIFEST.json  sentence_transformer.py  chunking_comparison.py  summarize_results.py  corpusManifest_converter.py  questions.yaml

57 token_splitter = TokenTextSplitter(chunk_size=256, chunk_overlap=50)
58 token_nodes = token_splitter.get_nodes_from_documents(documents)
59 token_index = VectorStoreIndex(token_nodes, embed_model=embed_model)
60
61 semantic_splitter = SemanticSplitterNodeParser(
62     buffer_size=1, embed_model=embed_model
63 )
64 semantic_nodes = semantic_splitter.get_nodes_from_documents(documents)
65 semantic_index = VectorStoreIndex(semantic_nodes, embed_model=embed_model)
66
67 sentence_window_parser = SentenceWindowNodeParser.from_defaults(
68     window_size=3,
69     window_metadata_key="window",
70     original_text_metadata_key="original_sentence",
71 )
72 sentence_nodes = sentence_window_parser.get_nodes_from_documents(documents)
73 sentence_index = VectorStoreIndex(sentence_nodes, embed_model=embed_model)
74
75
76 # 6. Helper Functions
77 def cosine_similarity(v1: np.ndarray, v2: np.ndarray) -> float:
78     return float(np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2)))
79
80
81 def evaluate_technique(q_item: dict, technique_name: str, index: VectorStoreIndex, k: int = 5):
82     query_id = q_item.get("id")
83     query_text = q_item.get("question")
84     expected_ans = q_item.get("expected_answer")
85     expected_src = q_item.get("expected_source")
86
87     query_emb = np.array(embed_model.get_query_embedding(query_text))
88     retriever = index.as_retriever(similarity_top_k=k)
89     results = retriever.retrieve(query_text)
90
91     records = []
92     for idx, result in enumerate(results, start=1):
93         content = result.node.get_content()
94         doc_emb = np.array(embed_model.get_text_embedding(content))
95         cos_sim = cosine_similarity(query_emb, doc_emb)
96         preview_text = content[:150].replace("\n", " ").strip() + "..."
97
98         records.append({
99             "question_id": query_id,
100             "query": query_text,
101             "expected_answer": expected_ans,
102             "expected_source": expected_src,
103             "technique": technique_name,
104             "rank": idx,
105             "store_score": round(result.score, 4) if result.score is not None else None,
106             "cosine_sim": round(cos_sim, 4),
107             "chunk_len": len(content),
108             "preview": preview_text
109         })
110
111     return records
112
113
```

```
112
113
114 # 7. Run Evaluations for All Questions across All Techniques
115 pipelines = [
116     ("Token", token_index),
117     ("Semantic", semantic_index),
118     ("Sentence-Window", sentence_index)
119 ]
120
121 all_records = []
122
123 for q_item in questions_data:
124     q_id = q_item.get("id")
125     q_text = q_item.get("question")
126
127     print("\n" + "#" * 100)
128     print(f"QUESTION ID: {q_id} | QUERY: {q_text}")
129     print("#" * 100)
130
131     for technique, index in pipelines:
132         records = evaluate_technique(q_item, technique, index, k=5)
133         all_records.extend(records)
134
135     # Print Output Table per question, per technique
136     df_technique = pd.DataFrame(records)[["rank", "store_score", "cosine_sim", "chunk_len", "preview"]]
137     print("\n" + "=" * 80)
138     print(f"TECHNIQUE: {technique}")
139     print("=" * 80)
140     print(df_technique.to_string(index=False))
141
142
143 # 8. Save Machine-Readable Output to raw/
144 csv_path = raw_dir / "retrieval_results.csv"
145 json_path = raw_dir / "retrieval_results.json"
146
147 df_all = pd.DataFrame(all_records)
148 df_all.to_csv(csv_path, index=False)
149
150 with open(json_path, "w", encoding="utf-8") as f:
151     json.dump(all_records, f, indent=2)
152
153 print(f"[+] Saved full evaluation dataset ({len(all_records)} total retrieval rows) to:\n - {csv_path}\n - {json_path}")
```

Summarize_results.py

```
Sublime Text  File  Edit  Selection  Find  View  Goto  Tools  Project  Window  Help
RUN LOG.txt  METRICS.md  verification.json  SOURCES.md  CORPUS_MANIFEST.json  sentence_transformer.py  chunking_comparison.py  summarize_results.py  corpusManifest_converter.py  questions.yaml
1  import pandas as pd
2  from pathlib import Path
3
4  # Paths Setup
5  script_dir = Path(__file__).resolve().parent
6  project_root = script_dir.parent
7  raw_csv = project_root / "raw" / "retrieval_results.csv"
8
9  if not raw_csv.exists():
10     raise FileNotFoundError(f"Raw data file non-existent at {raw_csv}. Run chunking_comparison.py first.")
11
12 # Load Raw Data
13 df = pd.read_csv(raw_csv)
14
15 print("\n-- Summary 1: Aggregate Metrics by Technique --")
16 print("RECOMPUTED SUMMARY TABLES FROM RAW DATA")
17 print("\n--")
18
19 # Summary 1: Metric Averages by Technique
20 summary_stats = df.groupby("technique").agg(
21     avg_cosine_sim=("cosine_sim", "mean"),
22     avg_store_score=("store_score", "mean"),
23     avg_chunk_len=("chunk_len", "mean"),
24     min_chunk_len=("chunk_len", "min"),
25     max_chunk_len=("chunk_len", "max"),
26     total_retrieved=("rank", "count")
27 ).reset_index()
28
29 print("\n-- Summary 1: Aggregate Metrics by Technique --")
30 print(summary_stats.to_string(index=False))
31
32 # Summary 2: Top Rank Comparison (Rank 1 Results)
33 top_rank_df = df[df["rank"] == 1][["technique", "store_score", "cosine_sim", "chunk_len", "preview"]]
34
35 print("\n-- Summary 2: Top-1 Rank Retrieval Comparison --")
36 print(top_rank_df.to_string(index=False))
```

```
Terminal  Shell  Edit  View  Window  Help
data260-1066 -- bash - 241x73
(venv) (base) Nathanael-MBP:data260-1866 nathanael python code/chunking_comparison.py
warning: The 'fitz' API is deprecated and will be removed in future. Use 'import pymupdf' instead.
[+] Loaded 1 document(s) with 24685 total characters.

=====
QUESTION ID: Q1 | QUERY: What caused SEPTA Trolley 111's pantograph to become entangled with the overhead catenary system on September 25, 2025?
=====

TECHNIQUE: Token
=====
rank  store_score  cosine_sim  chunk_len  preview
1      0.6787      0.6386      1021 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield,...
2      0.6148      0.4865      1189 and an operator onboard in Springfield, Pennsylvania, when its pantograph became entangled with the overhead catenary system (OCS).1 A portion of the pantogra...
3      0.5805      0.5983      1188 new OCS program and active, ongoing field oversight of OCS related activities. SEPTA's new formal OCS inspection and maintenance program was approved by PennD...
4      0.5988      0.5934      1283 SEPTA's maintenance records showed that on 47 occasions in the 14 months preceding the accident, a trolley experienced pantograph damage that required it to b...
5      0.5931      0.5638      1117 pantograph and contact wires and causing portions of the pantograph assembly to penetrate the trolley's passenger compartment. A trolley's pantograph usually...

TECHNIQUE: Semantic
=====
rank  store_score  cosine_sim  chunk_len  preview
1      0.6833      0.6568      1188 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield,...
2      0.6299      0.4895      993 The trolley operator and three passengers requested transport to a hospital for minor injuries. SEPTA's estimated damages to be about 1 (a) Visit ntab.gov to fi...
3      0.6476      0.5226      7837 Figure 1: Trolley 111 postaccident (left) and passenger compartment of Trolley 111 (right). Trolley 111, a single light rail vehicle, was traveling about 38 m...
4      0.4628      0.4422      8563 Railroad Investigation Report RIR-26-13 8 Traction power maintenance department. Employees became accustomed to frequent incidents of pantograph damage an...
5      0.4596      0.4767      6748 AC means alternating current; DC means direct current. 9 A catenary maintenance car is a specialized rail-bound vehicle used to inspect and repair overhead ca...

TECHNIQUE: Sentence-Window
=====
rank  store_score  cosine_sim  chunk_len  preview
1      0.7316      0.7788      567 Probable Cause The National Transportation Safety Board determines that the probable cause of the September 25, 2025, accident was Southeastern Pennsylvania T...
2      0.6856      0.6563      1892 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield,...
3      0.6865      0.5863      288 For over a year before the accident, SEPTA's vehicle maintenance department had been tracking an uptick in incidents on the D Line involving trolley pantograph...
4      0.5868      0.5698      263 Analysis Trolley 111's pantograph became entangled with sagging contact wires at the Scenic Road crossover, damaging the pantograph and contact wires and cau...
5      0.6741      0.5515      281 SEPTA's maintenance records showed that on 47 occasions in the 14 months preceding the accident, a trolley experienced pantograph damage that required it to b...

=====
QUESTION ID: Q2 | QUERY: How many pantograph damage incidents occurred on the D Line in the 14 months before the September 25, 2025 accident, and how did this compare with the previous long-term average?
=====

TECHNIQUE: Token
=====
rank  store_score  cosine_sim  chunk_len  preview
1      0.5878      0.5819      1203 SEPTA's maintenance records showed that on 47 occasions in the 14 months preceding the accident, a trolley experienced pantograph damage that required it to b...
2      0.5742      0.5684      1233 Inspection History and Pantograph Incident Trends SEPTA's traction power maintenance department had performed annual walking inspections of the D Line OCS. In...
3      0.4998      0.5184      1021 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield,...
4      0.4888      0.4842      1176 normalized and, therefore, did not raise sufficient concerns within the 10 See APTA Standard APTA-R7-PS-S-886-02, Traction Electrification Distribution Syste...
5      0.4882      0.4671      1195 (otherwise known as a practical or procedural drift)-a process by which a lack of controls allows a slow, gradual drift toward progressively unsafe practices...

TECHNIQUE: Semantic
=====
rank  store_score  cosine_sim  chunk_len  preview
1      0.4983      0.5874      1188 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield,...
2      0.4738      0.4669      8563 Railroad Investigation Report RIR-26-13 8 Traction power maintenance department. Employees became accustomed to frequent incidents of pantograph damage an...
3      0.4359      0.4212      993 The trolley operator and three passengers requested transport to a hospital for minor injuries. SEPTA's estimated damages to be about 1 (a) Visit ntab.gov to fi...
4      0.3752      0.5119      7837 Figure 1: Trolley 111 postaccident (left) and passenger compartment of Trolley 111 (right). Trolley 111, a single light rail vehicle, was traveling about 38 m...
5      0.2852      0.3128      6748 AC means alternating current; DC means direct current. 9 A catenary maintenance car is a specialized rail-bound vehicle used to inspect and repair overhead ca...

TECHNIQUE: Sentence-Window
=====
rank  store_score  cosine_sim  chunk_len  preview
1      0.6329      0.6388      288 For over a year before the accident, SEPTA's vehicle maintenance department had been tracking an uptick in incidents on the D Line involving trolley pantograph...
2      0.6313      0.6359      281 SEPTA's maintenance records showed that on 47 occasions in the 14 months preceding the accident, a trolley experienced pantograph damage that required it to b...
3      0.6164      0.5448      226 Because these small incidents did not result in significant negative consequences, employees came to accept both missed aerial inspections and regular inciden...
4      0.5799      0.5793      328 During the year before the accident, this increase of pantograph incidents became normalized and, therefore, did not raise sufficient concerns within the 10...
5      0.5781      0.4862      241 By 2024, vehicle maintenance records showed that SEPTA was experiencing incidents in which trolley pantographs were damaged during service about four times as...

=====
```

```
Terminal Shell Edit View Window Help
data260-1066 --bash --241x73

=====
TECHNIQUE: Token
=====
rank store_score cosine_sim chunk_len preview
1 0.7815 0.7124 1189 written OCS inspection and maintenance program, its missed monthly aerial inspections were not identified by SEPTA's management as inducing increased risk and...
2 0.6982 0.7973 1135 The actions taken by SEPTA in response to the audit findings did not address their root causes or the lack of a formal OCS inspection and maintenance program....
3 0.6739 0.6779 1167 Plan that details the agency's safety policies, objectives, and responsibilities utilizing an active safety management system framework. Railroad Investigati...
4 0.6635 0.6786 1137 at the location, interrupting power to the contact wire and de-energizing the OCS in the vicinity. After the accident, SEPTA halted passenger service on the...
5 0.6432 0.6382 1188 new OCS program and active, ongoing field oversight of OCS related activities. SEPTA's new formal OCS inspection and maintenance program was approved by PennDOT...

=====
TECHNIQUE: Semantic
=====
rank store_score cosine_sim chunk_len preview
1 0.6483 0.6487 8563 Railroad Investigation Report RIR-26-13 8 traction power maintenance department. Employees became accustomed to frequent incidents of pantograph damage an...
2 0.5977 0.6287 6748 AC means alternating current; DC means direct current. 9 A catenary maintenance car is a specialized rail-bound vehicle used to inspect and repair overhead ca...
3 0.5381 0.5282 1188 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield, Pa...
4 0.4889 0.4838 993 The trolley operator and three passengers requested transport to a hospital for minor injuries. SEPTA estimated damages to be about 1 (a) Visit ntsb.gov to fi...
5 0.3166 0.3124 7837 Figure 1: Trolley 111 postaccident (left) and passenger compartment of Trolley 111 (right). Trolley 111, a single light rail vehicle, was traveling about 38 m...

=====
TECHNIQUE: Sentence-Window
=====
rank store_score cosine_sim chunk_len preview
1 0.6764 0.6621 287 Evidence showed that SEPTA's traction power maintenance department likely experienced a normalization of deviance regarding its OCS inspection and maintenance...
2 0.6581 0.6831 598 However, the FTA does require SSOA to complete triennial audits of each rail transit agency's compliance with its Public Transportation Agency Safety Plan.6...
3 0.6341 0.6589 226 PennDOT SSOA's oversight failed to correct SEPTA's unsafe practices and lack of OCS procedures, even though PennDOT SSOA identified the inadequate inspection...
4 0.6115 0.6121 158 Inspection History and Pantograph Incident Trends. SEPTA's traction power maintenance department had performed annual walking inspections of the D Line OCS...
5 0.4885 0.4813 162 SEPTA's new formal OCS inspection and maintenance program was approved by PennDOT SSOA and the FTA and went into effect on April 28, 2026...

=====
QUESTION ID: Q4 | QUERY: What actions did SEPTA and federal and state safety agencies take after the Trolley 111 accident to improve overhead catenary system safety?
=====

=====
TECHNIQUE: Token
=====
rank store_score cosine_sim chunk_len preview
1 0.6845 0.6867 1188 new OCS program and active, ongoing field oversight of OCS related activities. SEPTA's new formal OCS inspection and maintenance program was approved by PennDOT...
2 0.6348 0.6328 1162 of the CAPs by SEPTA.7 CAPs should have addressed the underlying safety issues of missing and inconsistent OCS inspection plans and maintenance procedures tha...
3 0.6175 0.6289 1167 Plan that details the agency's safety policies, objectives, and responsibilities utilizing an active safety management system framework. Railroad Investigati...
4 0.5827 0.5588 1821 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield, Pa...
5 0.5964 0.5825 985 power the vehicle's motor. (d) An overhead catenary system uses an overhead wire to supply electricity to vehicles, such as light rail trains and streetcars. A...

=====
TECHNIQUE: Semantic
=====
rank store_score cosine_sim chunk_len preview
1 0.6372 0.5714 1188 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield, Pa...
2 0.5746 0.5762 993 The trolley operator and three passengers requested transport to a hospital for minor injuries. SEPTA estimated damages to be about 1 (a) Visit ntsb.gov to fi...
3 0.5485 0.5636 8563 Railroad Investigation Report RIR-26-13 8 traction power maintenance department. Employees became accustomed to frequent incidents of pantograph damage an...
4 0.5277 0.5244 6748 AC means alternating current; DC means direct current. 9 A catenary maintenance car is a specialized rail-bound vehicle used to inspect and repair overhead ca...
5 0.4898 0.4918 7837 Figure 1: Trolley 111 postaccident (left) and passenger compartment of Trolley 111 (right). Trolley 111, a single light rail vehicle, was traveling about 38 m...

=====
TECHNIQUE: Sentence-Window
=====
rank store_score cosine_sim chunk_len preview
1 0.6434 0.6528 567 Probable Cause The National Transportation Safety Board determines that the probable cause of the September 25, 2025, accident was Southeastern Pennsylvania T...
2 0.6258 0.5894 1092 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield, Pa...
3 0.5814 0.5991 456 Trolley 111, a single light rail vehicle, was traveling about 38 mph at the time of the accident, en route to Media, Pennsylvania.2 The D Line's OCS provided d...
4 0.5646 0.5646 288 For over a year before the accident, SEPTA's vehicle maintenance department had been tracking an uptick in incidents on the D Line involving trolley pantograph...
5 0.5641 0.5488 113 After the accident, SEPTA halted passenger service on the D Line to perform inspections and repairs to the OCS...

=====
QUESTION ID: Q5 | QUERY: According to the NTSB, what was the probable cause of the September 25, 2025 SEPTA Trolley 111 accident, and what was the contributing factor?
=====

=====
TECHNIQUE: Token
=====

=====
QUESTION ID: Q5 | QUERY: According to the NTSB, what was the probable cause of the September 25, 2025 SEPTA Trolley 111 accident, and what was the contributing factor?
=====

=====
TECHNIQUE: Semantic
=====
rank store_score cosine_sim chunk_len preview
1 0.5988 0.4576 1188 Railroad Investigation Report RIR-26-13 1 Southeastern Pennsylvania Transportation Authority Pantograph Separation From Trolley Car Location Springfield, Pa...
2 0.4417 0.4627 993 The trolley operator and three passengers requested transport to a hospital for minor injuries. SEPTA estimated damages to be about 1 (a) Visit ntsb.gov to fi...
3 0.4137 0.3842 8563 Railroad Investigation Report RIR-26-13 8 traction power maintenance department. Employees became accustomed to frequent incidents of pantograph damage an...
4 0.3888 0.3764 7837 Figure 1: Trolley 111 postaccident (left) and passenger compartment of Trolley 111 (right). Trolley 111, a single light rail vehicle, was traveling about 38 m...
5 0.3671 0.3525 6748 AC means alternating current; DC means direct current. 9 A catenary maintenance car is a specialized rail-bound vehicle used to inspect and repair overhead ca...

=====
TECHNIQUE: Sentence-Window
=====
rank store_score cosine_sim chunk_len preview
1 0.7118 0.6764 567 Probable Cause The National Transportation Safety Board determines that the probable cause of the September 25, 2025, accident was Southeastern Pennsylvania T...
2 0.5912 0.5785 241 By 2024, vehicle maintenance records showed that SEPTA was experiencing incidents in which trolley pantographs were damaged during service about four times as...
3 0.5811 0.5211 288 For over a year before the accident, SEPTA's vehicle maintenance department had been tracking an uptick in incidents on the D Line involving trolley pantograph...
4 0.5787 0.5686 251 SEPTA estimated damages to be about 1 (a) Visit ntsb.gov to find additional information in the public docket for this NTSB accident investigation (case number...
5 0.5296 0.4988 281 SEPTA's maintenance records showed that on 47 occasions in the 14 months preceding the accident, a trolley experienced pantograph damage that required it to b...

[+] Saved full evaluation dataset (75 total retrieval rows) to:
- /Users/nathanpak/Desktop/Data260/data260-1066/reports/hw83/raw/retrieval_results.csv
- /Users/nathanpak/Desktop/Data260/data260-1066/reports/hw83/raw/retrieval_results.json
(venv) (base) Nathan-MBP:data260-1066 nathanpak$
```

2. Observations

Sentence-window chunking performed best on this corpus because it optimizes vector representation while maintaining complete downstream context. By embedding single sentences (~205 characters on average), the vector model (all-MiniLM-L6-v2) evaluates dense, focused semantic units without vector dilution from surrounding sentences. During retrieval, attaching the adjacent context window (3 sentences) provides full explanatory background for answer generation without sacrificing vector precision. In contrast, Token chunking (~1,139 characters) frequently splits sentences arbitrarily across chunk boundaries, while Semantic

chunking produced overly large blocks (~3,515 characters) that diluted core facts, resulting in the lowest Top-1 similarity (0.5683).

Across all evaluated queries, Sentence-window chunking consistently achieved the highest Top-1 cosine similarity (0.6856) and highest Mean@5 cosine similarity (0.6075). While Token chunking occasionally retrieved useful context, its rigid character boundaries introduced background noise and split crucial technical facts. Semantic chunking failed to isolate granular answers cleanly due to its coarse boundary thresholding on this structured incident report.

3. Conclusion

Sentence-window chunking is the superior technique for this domain corpus. Supported by our empirical measurements, it achieved the highest Top-1 cosine similarity (0.6856 vs. 0.6327 for Token and 0.5683 for Semantic) while maintaining 100% Recall@5 at a low average retrieval latency of 9.74 ms. By isolating single sentences for similarity search while attaching neighboring context windows for LLM synthesis, Sentence-window chunking prevents context truncation across chunk boundaries and maximizes retrieval precision.